Cloud Computing Capstone: End-to-End Execution Guide

Complete Step-by-Step Runbook for Local Development & AWS EC2 Docker Swarm Production

Author: Chamroeun Saran **Course:** Master of Science in Cloud Computing **Repository:** github.com/Chamroeun12/Assignment

Target AWS: ap-southeast-2 (Sydney)

1. Project Architecture Overview

This repository provides a production-grade container orchestration and cloud delivery system on Amazon Web Services (AWS):

- **Infrastructure as Code (Terraform):** Provisions dedicated VPC (10.0.0.0/16), public subnets, least-privilege security groups, and 3 EC2 nodes (1 Manager + 2 Workers) with dynamic Canonical Ubuntu 22.04 LTS AMI discovery.
- **Container Orchestration (Docker Swarm):** 3-node cluster operating behind an overlay network with internal service VIP routing and zero-downtime rolling updates (`order: start-first`).
- **Ingress & Load Balancing (Nginx):** Reverse proxy on Port 80 terminating public traffic and balancing requests across replicated Python Flask containers.
- **CI/CD Automation (Jenkins):** Declarative pipeline running Flake8 linting, Pytest test suites, Trivy CVE vulnerability scans, and Telegram Bot push notifications.

2. System Prerequisites

Tool / Requirement	Local Testing Mode	AWS Cloud Mode	Recommended Version
Git	Required	Required	2.30+
Docker Desktop	Required	Optional	24.0+ (Docker Compose v2)
AWS CLI	Not Needed	Required	2.10+ (configured with access keys)
Terraform CLI	Not Needed	Required	1.5.0+
SSH Key Pair	Not Needed	Required	`chamroeunsaran.pem` (400 permissions)

3. Method A: Quick Local Testing with Docker Compose (No AWS Required)

If you do not have an AWS account and want to test the full multi-service application (Flask App + Nginx Load Balancer) locally on your laptop in 30 seconds:

Step 3.1: Clone Repository

```
git clone https://github.com/Chamroeun12/Assignment.git
cd Assignment
```

Step 3.2: Build and Launch Multi-Service Stack

```
docker compose up --build -d
```

Step 3.3: Verify Running Services

```
docker compose ps
```

Both `cicd-assignment-web` (Flask Backend on port 5000) and `cicd-assignment-nginx` (Reverse Proxy on port 80) will be running and healthy.

Step 3.4: Test Endpoints in Browser or Terminal

Open your browser at: `http://localhost/`

```
curl -i http://localhost/            # Main Web UI Dashboard
curl -s http://localhost/health       # JSON Healthcheck
curl -s http://localhost/api/info     # Backend Service & Hostname
curl -s http://localhost/nginx-health # Nginx Proxy Health
```

Step 3.5: Clean Up Local Containers

```
docker compose down
```

4. Method B: Full Production Deployment on AWS EC2 (Terraform & Docker Swarm)

Follow these steps to provision your own 3-node cluster on AWS EC2 and deploy the production Docker Swarm stack:

Step 4.1: Configure AWS CLI

```
aws configure
# AWS Access Key ID: [Your Access Key]
# AWS Secret Access Key: [Your Secret Key]
# Default region: ap-southeast-2
# Default output format: json
```

Step 4.2: Provision Infrastructure with Terraform

```
cd terraform
terraform init
terraform plan
terraform apply -auto-approve
```

Terraform outputs the generated public and private IPs:

```
Outputs:
manager_public_ip = "16.178.206.23"
manager_private_ip = "172.31.34.160"
worker_public_ips = ["3.107.96.140", "13.211.213.228"]
worker_private_ips = ["172.31.36.22", "172.31.47.183"]
application_url = "http://16.178.206.23"
```

Step 4.3: Initialize Docker Swarm & Connect Nodes

1. SSH into the Manager Node:

```
ssh -i chamroeunsaran.pem ubuntu@<manager_public_ip>
```

2. Initialize Docker Swarm on Manager:

```
sudo docker swarm init --advertise-addr <manager_private_ip>
sudo hostnamectl set-hostname swarm-manager
sudo systemctl restart docker
```

Copy the worker join token command printed on screen (`docker swarm join --token SWMTKN-1-...`).

3. Join Worker 1 to Swarm:

```
ssh -i chamroeunsaran.pem ubuntu@<worker1_public_ip>
sudo docker swarm join --token <TOKEN> <manager_private_ip>:2377
sudo hostnamectl set-hostname swarm-worker1
sudo systemctl restart docker
exit
```

4. Join Worker 2 to Swarm:

```
ssh -i chamroeunsaran.pem ubuntu@<worker2_public_ip>
sudo docker swarm join --token <TOKEN> <manager_private_ip>:2377
sudo hostnamectl set-hostname swarm-worker2
sudo systemctl restart docker
exit
```

5. Verify Swarm Topology on Manager:

```
sudo docker node ls
# ID            HOSTNAME        STATUS    AVAILABILITY   MANAGER STATUS
# xxxxxxxx *    swarm-manager   Ready     Active         Leader
# yyyyyyyy      swarm-worker1   Ready     Active
# zzzzzzzz      swarm-worker2   Ready     Active
```

Step 4.4: Deploy Application Stack

Copy code bundle to the cluster nodes and deploy:

```
# Package from local workstation:
tar -czf app_bundle.tar.gz Dockerfile app.py requirements.txt templates docker-stack.yml nginx

# SCP to all 3 nodes:
scp -i chamroeunsaran.pem app_bundle.tar.gz ubuntu@<manager_public_ip>:/home/ubuntu/
scp -i chamroeunsaran.pem app_bundle.tar.gz ubuntu@<worker1_public_ip>:/home/ubuntu/
scp -i chamroeunsaran.pem app_bundle.tar.gz ubuntu@<worker2_public_ip>:/home/ubuntu/

# Build image on nodes:
ssh -i chamroeunsaran.pem ubuntu@<worker1_public_ip> "tar -xzf app_bundle.tar.gz && sudo docker build -t 399
ssh -i chamroeunsaran.pem ubuntu@<worker2_public_ip> "tar -xzf app_bundle.tar.gz && sudo docker build -t 399

# Deploy on Manager:
ssh -i chamroeunsaran.pem ubuntu@<manager_public_ip>
tar -xzf app_bundle.tar.gz
sudo docker build -t 399311815210.dkr.ecr.ap-southeast-2.amazonaws.com/cicd-assignment:latest .
sudo docker stack deploy -c docker-stack.yml cicd-assignment
```

Step 4.5: Verify Services & Live Endpoints

```
sudo docker service ls
sudo docker stack ps cicd-assignment
```

Test the live endpoints via curl:

```
curl -s http://<manager_public_ip>/health
curl -s http://<manager_public_ip>/api/info
curl -s http://<manager_public_ip>/nginx-health
```

5. Method C: Automated Jenkins CI/CD Setup

To run our pre-configured Jenkins environment locally with all plugins, credentials, and build history:

```
# 1. Start Jenkins container
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home jenkins/jenkins:l

# 2. Inject backup archive
docker cp jenkins_backup.tar.gz jenkins:/tmp/jenkins_backup.tar.gz
docker exec -u 0 jenkins tar xzf /tmp/jenkins_backup.tar.gz -C /var/jenkins_home
docker exec -u 0 jenkins chown -R jenkins:jenkins /var/jenkins_home

# 3. Disable authentication for instant presentation access
docker exec -u 0 jenkins sed -i 's/<useSecurity>true<\/useSecurity>/<useSecurity>false<\/useSecurity>/' /var
docker restart jenkins

# 4. Open in browser:
http://localhost:8080/job/Assignment-CI-CD/
```

6. Automated Testing & Linting

Run the test suite and PEP8 linter on your workstation:

```
# Create virtual environment
python -m venv .venv
source .venv/bin/activate  # On Windows: .venv\Scripts\Activate.ps1

# Install requirements
pip install -r requirements.txt
pip install -r requirements-dev.txt

# Run Flake8 linter
flake8 --max-line-length=120 --exclude=venv,.venv,build_submission_report.py,generate_evidence_screens.py ap

# Run Pytest suite with coverage
pytest -v --cov=app --cov-report=term-missing tests/
```

Result: 5/5 tests passed with 89% statement coverage.

7. Cost Management & Teardown

> **Important:** AWS charges hourly for running EC2 instances and attached EBS volumes. Remember to stop or destroy instances when finished!

To Pause Instances (Stops Compute Billing, Keeps Data Intact):

```
aws ec2 stop-instances --instance-ids <mgr_id> <w1_id> <w2_id> --region ap-southeast-2
```

To Permanently Destroy All AWS Resources:

```
cd terraform
terraform destroy -auto-approve
```

8. Frequently Asked Questions (FAQ)

- **Q: Why did my public IP change after stopping EC2?**
  AWS dynamic public IPs are released back into the Amazon pool upon instance stop. Internal private IPs remain fixed, keeping the internal Swarm cluster intact.

- **Q: How do I test the zero-downtime rolling update?**
  Execute: `sudo docker service update --image <IMAGE>:v2 --update-order start-first cicd-assignment_web`.
  Swarm provisions healthy new containers before stopping old ones.
- **Q: How do I view live application logs?**
  Run: `sudo docker service logs -f cicd-assignment_web`.